

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

Exercises: math and datetime modules

Introduction to Computer Programming

Management and Artificial Intelligence

LUISS



math module

sqrt()

```
import math  
  
number = 25  
sqrt_value = math.sqrt(number)  
print(f"The square root of {number} is {sqrt_value}")
```

The square root of 25 is 5.0

pi and sin()

```
angle_in_radians = math.pi / 6 # 30 degrees
sine_value = math.sin(angle_in_radians)
print(f"Sine of 30 degrees (in radians) is {sine_value}")
```

Sine of 30 degrees (in radians) is 0.49999999999999994

factorial()

```
num = 5
fact_value = math.factorial(num)
print(f"The factorial of {num} is {fact_value}")
```

The factorial of 5 is 120

log()

```
number = 10
natural_log = math.log(number)
print(f"Natural logarithm of {number} is {natural_log}")
```

Natural logarithm of 10 is 2.302585092994046

pow()

```
base = 2
exponent = 3
power_value = math.pow(base, exponent)
print(f"{base} raised to the power of {exponent} is {power_value}")
```

2 raised to the power of 3 is 8.0

gcd()

```
num1 = 48  
num2 = 180  
gcd_value = math.gcd(num1, num2)  
print(f"The GCD of {num1} and {num2} is {gcd_value}")
```

The GCD of 48 and 180 is 12

degrees ()

```
radians_value = math.pi  
degrees_value = math.degrees(radians_value)  
print(f"{radians_value} radians is equal to {degrees_value} degrees")
```

3.141592653589793 radians is equal to 180.0 degrees

hypot()

```
side_a = 3
side_b = 4
hypotenuse = math.hypot(side_a, side_b)
print(
    f"The hypotenuse of a right triangle with sides {side_a} and {side_b} is {hypotenuse}"
)
```

The hypotenuse of a right triangle with sides 3 and 4 is 5.0

ceil()

```
num = 4.3  
ceil_value = math.ceil(num)  
print(f"The ceiling of {num} is {ceil_value}")
```

The ceiling of 4.3 is 5

floor()

```
num = 4.8  
floor_value = math.floor(num)  
print(f"The floor of {num} is {floor_value}")
```

The floor of 4.8 is 4

Exercises

- Notebook with exercises on math (no hints)
- Notebook with exercises on math (with pseudocode)

`datetime` module

now()

```
from datetime import datetime  
  
now = datetime.now()  
print("Current date and time:", now)
```

Current date and time: 2024-09-17 21:03:51.704489

today()

```
today = datetime.today()  
print("Year:", today.year)  
print("Month:", today.month)  
print("Day:", today.day)
```

Year: 2024

Month: 9

Day: 17

datetime()

```
specific_date = datetime(2024, 12, 25, 15, 30)  
print("Specific date and time:", specific_date)
```

Specific date and time: 2024-12-25 15:30:00

strftime()

```
formatted_date = today.strftime("%Y-%m-%d %H:%M:%S")  
print("Formatted date:", formatted_date)
```

Formatted date: 2024-09-17 21:03:51

strptime()

```
date_string = "2024-09-17 14:30:00"  
parsed_date = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")  
print("Parsed date:", parsed_date)
```

Parsed date: 2024-09-17 14:30:00

timedelta()

```
from datetime import timedelta

future_date = today + timedelta(days=10)
past_date = today - timedelta(days=5)
print("Date 10 days from now:", future_date)
print("Date 5 days ago:", past_date)
```

Date 10 days from now: 2024-09-27 21:03:51.715959

Date 5 days ago: 2024-09-12 21:03:51.715959

datetime subtraction

```
date1 = datetime(2023, 9, 1)
date2 = datetime(2024, 9, 1)
difference = date2 - date1
print("Difference between two dates:", difference.days, "days")
```

Difference between two dates: 366 days

time()

```
current_time = now.time()  
print("Current time:", current_time)
```

Current time: 21:03:51.704489

Check if an year is a leap year

```
year = 2024
is_leap = year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
print(f"Is {year} a leap year?", is_leap)
```

Is 2024 a leap year? True

weekday()

```
weekday = today.weekday() # Monday = 0, Sunday = 6  
print(f"Today is day {weekday} of the week (0=Monday, 6=Sunday)")
```

Today is day 1 of the week (0=Monday, 6=Sunday)

Exercises

- [Notebook with exercises on datetime \(no hints\)](#)
- [Notebook with exercises on datetime \(with pseudocode\)](#)

